

Diffusion-LM: 제어 가능한 텍스트 생성을 향상시키다

(Diffusion-LM Improves Controllable Text Generation — 한국어 PDF판)

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, Tatsunori B. Hashimoto (Stanford University)
한국어 번역: Compendium (<https://yoon-gu.github.io/compendium/>)

Contents

0.1	초록	2
0.2	1. 서론	2
0.3	2. 관련 연구	3
0.4	3. 문제 정의 및 배경	4
0.4.1	3.1 텍스트의 생성 모델과 제어 가능 생성	4
0.4.2	3.2 자기회귀 언어모델	4
0.4.3	3.3 연속 영역의 디퓨전 모델	4
0.5	4. Diffusion-LM: 연속 디퓨전 언어모델링	5
0.5.1	4.1 엔드투엔드 학습	5
0.5.2	4.2 라운딩 오류 줄이기	6
0.6	5. Diffusion-LM 의 디코딩과 제어 가능 생성	6
0.6.1	5.1 제어 가능 텍스트 생성	6
0.6.2	5.2 최소 베이스 위험 디코딩	7
0.7	6. 실험 설정	8
0.7.1	6.1 데이터셋과 하이퍼파라미터	8
0.7.2	6.2 제어 과제	8
0.7.3	6.3 분류기 가이드된 베이스라인	8
0.7.4	6.4 인필링 베이스라인	9
0.8	7. 주요 결과	9
0.8.1	7.1 분류기 가이드된 제어 가능 텍스트 생성 결과	9
0.8.2	7.2 제어 합성	10
0.8.3	7.3 인필링 결과	10
0.8.4	7.4 절제 연구	11
0.9	8. 결론과 한계	11
0.10	부록 발췌 (요약)	11

원문: [Diffusion-LM Improves Controllable Text Generation](#) — Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, Tatsunori B. Hashimoto (Stanford University), NeurIPS 2022, arXiv 2205.14217

아래 글은 원문 논문의 순서와 서술을 최대한 따라가며 한국어로 옮긴 것이다. 가능한 한 문장 단위 대응을 유지하되, 한국어 문장만 최소한으로 다듬었다.

0.1 초록

언어모델(language model, LM)의 동작을 재학습 없이 제어하는 일은 자연어 생성(natural language generation)에서 풀리지 않은 큰 문제 중 하나다. 최근 연구들은 감성(sentiment) 같은 단순한 문장 속성 제어에서는 성공을 보였지만, 구문 구조(syntactic structure) 같은 복잡하고 세밀한 제어에서는 진전이 거의 없었다. 이 도전에 답하기 위해 저자들은 연속(continuous) 디퓨전에 기반한 새로운 비자기회귀(non-autoregressive) 언어모델인 Diffusion-LM 을 개발한다. 연속 영역에서 디퓨전 모델이 거둔 최근의 성공을 발판 삼아, Diffusion-LM 은 가우시안 벡터 시퀀스를 단어 벡터로 반복적으로 잡음제거하며 그 과정에서 일련의 중간 잠재 변수(latent variable)를 만들어 낸다. 이 중간 잠재의 연속적이고 위계적인 성격 덕분에, 단순한 그라디언트 기반 알고리즘으로 복잡한 제어 가능 생성 과정을 수행할 수 있다. 저자들은 6개의 까다로운 세밀 제어(fine-grained control) 과제에서 Diffusion-LM 의 성공적 제어를 보이며, 기존 연구를 큰 폭으로 능가한다. 코드는 github.com/XiangLi1999/Diffusion-LM 에서 제공한다.

0.2 1. 서론

대규모 자기회귀(autoregressive, AR) 언어모델은 고품질 텍스트를 생성할 수 있지만, 실제 응용에 신뢰성 있게 배포하려면 텍스트 생성 과정이 제어 가능해야 한다 — 즉 원하는 요건(주제, 구문 구조 등)을 만족하는 텍스트를 생성할 수 있어야 한다. LM 제어의 자연스러운 접근 하나는 (제어, 텍스트) 쌍의 지도 데이터로 LM 을 파인튜닝(fine-tuning) 하는 것이다. 그러나 제어 과제마다 LM 파라미터를 갱신하는 일은 비싸고, 여러 제어를 합성(composition) 할 수 없다 (예: 긍정 감성 그리고 비유해성 텍스트 동시 생성). 이는 가벼우면서 모듈식인 플러그 앤 플레이(plug-and-play) 접근의 동기가 된다. 이런 접근은 LM 을 동결한 채 외부 분류기(classifier)로 생성 과정을 조향하는데, 분류기는 생성된 텍스트가 제어를 얼마나 잘 만족하는지 측정한다. 그러나 동결된 자기회귀 LM 을 조향하는 일은 어려움이 알려져 있고, 기존 성공 사례는 대부분 속성 수준 의 단순 제어(감성·주제 등)에 한정되었다.

더 복잡한 제어를 다루기 위해 저자들은 연속 디퓨전에 기반한 새 언어모델 Diffusion-LM 을 제안한다. Diffusion-LM 은 가우시안 잡음 벡터 시퀀스에서 출발해 그것을 점진적으로 단어에 대응되는 벡터로 잡음제거(denoise) 한다 (그림 1). 이 점진적 잡음제거 단계는 연속 잠재 표현의 위계를 만들어 내며, 이 위계적·연속적 잠재 변수가 있어야 비로소 단순한 그라디언트 기반 방법으로 생성 시퀀스의 파스 트리(parse tree) 제약 같은 복잡한 제어 과제를 풀 수 있다.

연속 디퓨전 모델은 시각·오디오 영역에서 큰 성공을 거두었지만, 텍스트의 본질적 이산성(discreteness) 때문에 텍스트에는 거의 적용되지 않았다 (§ 배경). 이 부류의 모델을 텍스트에 맞추려면 표준 디퓨전에 몇 가지 수정이 필요하다 — 표준 디퓨전 과정에 임베딩 단계와 라운딩 단계를 더하고, 임베딩을 학습하는 학습 목적함수를 설계하며, 라운딩을 개선하는 기법을 제안한다 (§ 방법). Diffusion-LM 의 제어는 그라디언트 기반 방법으로 수행한다 (그림 1). 이 방법으로 우리는 텍스트 생성 과정을 목표 구조·의미 제약을 만족하는 출력으로 조향할 수 있다.

구체적으로는 Diffusion-LM 의 연속 잠재 변수에 대해 유창성과 제어 만족도의 균형을 맞추도록 반복적으로 그라디언트 업데이트를 수행한다 (§ 제어 텍스트 생성).

Diffusion-LM 의 제어를 보이기 위해, 저자들은 세밀 속성 제어부터 복잡 구조 제어에 이르는 6개 제어 표적을 검토한다. 본 방법은 기존 플러그애플레이 방법의 성공률을 거의 두 배로 끌어올리고, 분류기 가이드를 기반 모든 과제에서 파인튜닝 오라클(oracle)과 동등하거나 더 나은 결과를 보인다 (§ 제어). 개별 제어 과제 외에도, 다중 분류기 가이드를 합성하여 원하는 의미 콘텐츠와 구문 구조를 동시에 갖춘 문장을 생성할 수 있음을 보인다 (§ 합성). 마지막으로 길이 제어와 인필링(infilling) 같은 스패 고정 제어(span-anchored controls)도 다룬다. Diffusion-LM 은 이런 과제를 분류기 없이 수행할 수 있고, 인필링에서 처음부터 학습한 자기회귀 LM 과 동등하면서 기존 플러그애플레이 방법을 큰 폭 능가한다 (§ 인필링).

0.3 2. 관련 연구

텍스트용 디퓨전 모델. 디퓨전 모델은 연속 데이터 영역에서 큰 성공을 거두며 최첨단 표본 품질의 이미지·오디오를 만들어 왔다. 이산 데이터를 다루기 위해 과거 연구들은 이산 상태 공간에서의 텍스트 디퓨전 모델을 연구해 왔으며, 이산 데이터에 대한 부패 과정(corruption process) — 예컨대 각 토큰이 흡수 토큰이나 임의 토큰으로 부패될 일정 확률 — 을 정의한다. 본 논문은 연속 디퓨전 모델에 초점을 두며, 저자들이 아는 한 이 설정을 탐구한 첫 작업이다. 이산 디퓨전 LM 과 달리 연속 디퓨전 LM 은 연속 잠재 표현을 유도하므로, 효율적인 그라디언트 기반 제어가 가능해진다.

자기회귀 vs 비자기회귀 LM. 대부분의 대형 사전학습 LM 은 좌→우 자기회귀(GPT-3, PaLM 등)다. 고정된 생성 순서는 많은 제어 가능 생성 설정 — 특히 좌·우 양쪽 컨텍스트에 전역 제약을 걸어야 하는 경우 — 에서 모델 유연성을 제약한다. 한 예가 인필링으로, 우측 컨텍스트에 어휘 제약을 부과한다; 또 다른 예가 구문 구조 제어로, 좌·우 양쪽이 관여하는 전역 속성을 제어한다. 자기회귀 LM 은 우측 컨텍스트에 직접 조건화할 수 없으므로, 기존 연구들은 이런 과제를 위한 특수 학습·디코딩 기법을 개발해 왔다. 예를 들어 [Qin-COLD] 는 이산 LM 출력을 연속 변수로 완화한 뒤 우측 컨텍스트로부터 그라디언트를 역전파하는 디코딩 방법을 제안했다. Diffusion-LM 은 문장의 복잡·전역 속성을 보는 임의의 분류기에 조건화할 수 있다. 비자기회귀 LM 은 기계 번역과 음성-텍스트 과제용으로도 개발되어 왔지만, 이들 방법은 출력 엔트로피가 낮은 음성·번역 설정에 특화되어 있고 언어모델링에는 실패함이 알려져 있다.

플러그애플레이 제어 가능 생성. 플러그애플레이 제어는 LM 을 동결한 채 잠재 함수(예: 분류기)로 출력을 조향한다. 생성 텍스트가 원하는 제어를 얼마나 잘 만족하는지 측정하는 확률적 잠재 함수가 주어지면, 생성 텍스트는 잠재 함수로 측정되는 제어 만족도와 LM 확률로 측정되는 유창성을 동시에 최적화하도록 만들어진다. 자기회귀 LM 기반 플러그애플레이 접근으로는 FUDGE — 부분 시퀀스의 제어 만족도 추정으로 매 토큰의 LM 예측을 재가중 — 와 GeDi·DExperts — 더 작은 LM (제어 과제용 학습/파인튜닝) 으로 매 토큰을 재가중 — 가 있다.

본 연구와 가장 가까운 작업은 PPLM 으로, 자기회귀 LM 의 은닉 활성화에 대해 그라디언트 상승을 수행해 다음 토큰이 제어를 만족하면서도 유창성을 유지하도록 조향한다. PPLM 은 자기회귀 LM 기반이므로 좌→우로만 생성할 수 있고, 이 때문에 이전 단계의 오류를 복구하지 못한다. 속성(예: 주제) 제어에서 성공함에도 불구하고, 자기회귀 LM 용 플러그애플레이 방법들은 구문 구조나 의미 콘텐츠 같은 복잡 제어에서는 실패함을 § 제어에서 보일 것이다. Diffusion-LM 은 이로부터 유도되는 연속 잠재 변수 시퀀스에 분류기 가이드를 그라디언트 업데이트를 적용하여 플러그애플레이 제어 가능 생성을 수행할 수 있음을 보인다.

0.4 3. 문제 정의 및 배경

먼저 제어 가능 생성을 정의하고 (§ 3.1), 연속 디퓨전 모델을 개관한다 (§ 3.3).

0.4.1 3.1 텍스트의 생성 모델과 제어 가능 생성

텍스트 생성은 학습된 언어모델 $p_{\text{lm}}(\mathbf{w})$ 로부터 $\mathbf{w}$ 를 표집하는 과제다. 여기서 $\mathbf{w} = [w_1 \dots w_n]$ 은 이산 단어 시퀀스이고 $p_{\text{lm}}(\mathbf{w})$ 는 단어 시퀀스 위의 확률 분포다. 제어 가능 텍스트 생성은 조건부 분포 $p(\mathbf{w} | c)$ 로부터 $\mathbf{w}$ 를 표집하는 과제로, c 는 제어 변수(control variable)다. 구문 제어에서 c 는 목표 구문 트리, 감성 제어에서 c 는 원하는 감성 라벨 등이 될 수 있다. 목표는 제어 표적 c 를 만족하는 $\mathbf{w}$ 를 생성하는 것이다.

플러그앤플레이 설정에서 우리는 (1) 라벨 없는 대량 텍스트로부터 학습된 LM $p_{\text{lm}}(\mathbf{w})$, (2) 각 제어 과제별로 더 적은 라벨 텍스트로 학습된 분류기 $p(c | \mathbf{w})$ (구문 제어라면 확률적 파서)를 갖는다. 두 모델을 활용해 베이지 규칙 $p(\mathbf{w} | c) \propto p_{\text{lm}}(\mathbf{w}) \cdot p(c | \mathbf{w})$ 를 통해 사후 분포에서 근사적으로 표집하는 것이 목표다. 여기서 $p_{\text{lm}}(\mathbf{w})$ 가 $\mathbf{w}$ 의 유창성을, $p(c | \mathbf{w})$ 가 제어 만족을 유도한다.

0.4.2 3.2 자기회귀 언어모델

LM의 정형적 접근은 p_{lm} 을 좌→우 자기회귀로 분해한다.

$$p_{\text{lm}}(\mathbf{w}) = p_{\text{lm}}(w_1) \prod_{i=2}^n p_{\text{lm}}(w_i | w_{<i}).$$

이 경우 텍스트 생성은 부분 시퀀스에 조건화된 다음 토큰 예측의 반복으로 환원된다. 다음 토큰 예측 $p_{\text{lm}}(w_i | w_{<i})$ 는 흔히 트랜스포머(Transformer) 아키텍처로 매개변수화된다.

0.4.3 3.3 연속 영역의 디퓨전 모델

디퓨전 모델은 데이터 $\mathbf{x}_0 \in \mathbb{R}^d$ 를 마르코프 체인 $\mathbf{x}_T \dots \mathbf{x}_0$ (각 변수 $\in \mathbb{R}^d$, $\mathbf{x}_T$ 는 가우시안)으로 모델링하는 잠재 변수 모델이다. 디퓨전 모델은 잠재 시퀀스 $\mathbf{x}_{T:1}$ 을 점진적으로 잡음제거해 목표 데이터 분포의 표본을 근사한다 (그림 2). 초기 상태 $p_{\theta}(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$ 이고, 잡음제거 전이 $\mathbf{x}_t \rightarrow \mathbf{x}_{t-1}$ 은

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

로 매개변수화된다. $\mu_{\theta}, \Sigma_{\theta}$ 는 U-Net 또는 트랜스포머로 계산할 수 있다.

학습을 위해서는 중간 잠재 $\mathbf{x}_{1:T}$ 를 만드는 forward 과정을 정의한다. forward는 데이터 $\mathbf{x}_0$ 에 가우시안 노이즈를 점진적으로 더해, 디퓨전 단계 T 에서 $\mathbf{x}_T$ 가 근사적으로 가우시안이 되도록 한다. 각 전이는

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}),$$

로 매개변수화되며, β_t 는 단계 t 에서 더해지는 노이즈 양이다. forward q 는 학습 파라미터를 갖지 않으며, 사전 정의된 forward로 노이즈 데이터를 만들고 모델이 그 과정을 역으로 데이터를 복원하도록 학습하는 목적함수를 정의할 수 있게 해 준다.

디퓨전 모델은 데이터의 주변 우도 $\mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}} [\log p_{\theta}(\mathbf{x}_0)]$ 를 최대화하도록 학습되며, 정형 목적함수는 $\log p_{\theta}(\mathbf{x}_0)$ 의 변분 하한이다.

$$\mathcal{L}_{\text{vib}}(\mathbf{x}_0) = \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_T | \mathbf{x}_0)}{p_{\theta}(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_0, \mathbf{x}_t)}{p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)} - \log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1) \right].$$

그러나 이 목적함수는 불안정하여 안정화에 많은 트릭이 든다. 이를 우회하기 위해 [ho2020denoising] 은 $\mathcal{L}_{\text{vib}}$ 의 각 KL 항을 펼쳐 재가중한 평균제곱오차 손실 $\mathcal{L}_{\text{simple}}$ (유도는 부록) 을 고안했다.

$$\mathcal{L}_{\text{simple}}(\mathbf{x}_0) = \sum_{t=1}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \|\mu_{\theta}(\mathbf{x}_t, t) - \hat{\mu}(\mathbf{x}_t, \mathbf{x}_0)\|^2,$$

여기서 $\hat{\mu}(\mathbf{x}_t, \mathbf{x}_0)$ 는 가우시안 닫힌 형태인 사후 $q(\mathbf{x}_{t-1} | \mathbf{x}_0, \mathbf{x}_t)$ 의 평균, $\mu_{\theta}(\mathbf{x}_t, t)$ 는 신경망이 예측하는 $p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 의 평균이다. $\mathcal{L}_{\text{simple}}$ 은 더 이상 유효한 변분 하한은 아니지만, 학습을 안정화하고 표본 품질을 높이는 것이 경험적으로 확인되었다. Diffusion-LM 도 학습 안정화와 표본 품질을 위해 비슷한 단순화를 사용한다 (§ 4.1).

0.5 4. Diffusion-LM: 연속 디퓨전 언어모델링

Diffusion-LM 을 만들려면 표준 디퓨전 모델에 몇 가지 수정이 필요하다. 첫째, 이산 텍스트를 연속 공간으로 보내는 임베딩 함수가 필요하다. 이를 위해 임베딩을 학습하는 엔드투엔드 학습 목적함수를 제안한다 (§ 4.1). 둘째, 임베딩 공간의 벡터를 다시 단어로 보내는 라운딩 방법이 필요하다. 이를 위해 학습·디코딩 시 라운딩을 돕는 방법을 제안한다 (§ 4.2).

0.5.1 4.1 엔드투엔드 학습

연속 디퓨전을 이산 텍스트에 적용하기 위해, 각 단어를 $\mathbb{R}^d$ 의 벡터로 보내는 임베딩 함수 $\text{Emb}(w_i)$ 를 정의한다. 길이 n 시퀀스 $\mathbf{w}$ 의 임베딩은 $\text{Emb}(\mathbf{w}) = [\text{Emb}(w_1), \dots, \text{Emb}(w_n)] \in \mathbb{R}^{nd}$.

저자들은 디퓨전 모델 학습 목적함수를 임베딩과 디퓨전 파라미터를 함께 학습하도록 수정한다. 예비 실험에서 무작위 가우시안 임베딩과 사전학습 단어 임베딩(GloVe, GPT-2 등)을 모두 시험했고, 이런 고정 임베딩은 엔드투엔드 학습보다 Diffusion-LM 에서 차선임을 발견했다.

그림 2 처럼, 본 접근은 forward 과정에 이산 단어 $\mathbf{w}$ 와 $\mathbf{x}_0$ 사이의 마르코프 전이를 추가한다.

$$q_{\phi}(\mathbf{x}_0 | \mathbf{w}) = \mathcal{N}(\text{Emb}(\mathbf{w}), \sigma_0 I).$$

역과정에는 학습 가능한 라운딩 단계 $p_{\theta}(\mathbf{w} | \mathbf{x}_0) = \prod_{i=1}^n p_{\theta}(w_i | x_i)$ 를 추가한다 ($p_{\theta}(w_i | x_i)$ 는 소프트맥스 분포). 학습 목적함수는 다음과 같이 된다.

$$\begin{aligned} \mathcal{L}_{\text{vib}}^{\text{e2e}}(\mathbf{w}) &= \mathbb{E}_{q_{\phi}(\mathbf{x}_0|\mathbf{w})} [\mathcal{L}_{\text{vib}}(\mathbf{x}_0) + \log q_{\phi}(\mathbf{x}_0 | \mathbf{w}) - \log p_{\theta}(\mathbf{w} | \mathbf{x}_0)], \\ \mathcal{L}_{\text{simple}}^{\text{e2e}}(\mathbf{w}) &= \mathbb{E}_{q_{\phi}(\mathbf{x}_{0:T}|\mathbf{w})} [\mathcal{L}_{\text{simple}}(\mathbf{x}_0) + \|\text{Emb}(\mathbf{w}) - \mu_{\theta}(\mathbf{x}_1, 1)\|^2 - \log p_{\theta}(\mathbf{w} | \mathbf{x}_0)]. \end{aligned}$$

$\mathcal{L}_{\text{simple}}^{\text{e2e}}$ 은 $\mathcal{L}_{\text{vlb}}^{\text{e2e}}$ 에서 § 3.3 의 단순화를 따라 유도한다 (부록 참조). 임베딩 함수가 학습되므로 q_ϕ 가 학습 파라미터를 갖고, 표집 단계의 역전파에는 재매개변수화 트릭(reparametrization trick)을 쓴다. 경험적으로 학습된 임베딩은 의미 있게 군집된다 — 같은 품사(POS, 구문 역할) 단어들이 함께 묶이는 경향을 보인다 (그림 3).

0.5.2 4.2 라운딩 오류 줄이기

학습된 임베딩은 이산 텍스트를 연속 $\mathbf{x}_0$ 로 보내는 매핑을 정의한다. 이제 예측된 $\mathbf{x}_0$ 를 다시 이산 텍스트로 라운딩(rounding) 하는 역과정을 설명한다. 라운딩은 위치별로 가장 그럴듯한 단어를 고르는 것 — $\text{argmax } p_\theta(\mathbf{w} | \mathbf{x}_0) = \prod_{i=1}^n p_\theta(w_i | x_i)$ — 으로 수행한다. 이상적으로는 잡음제거 단계가 $\mathbf{x}_0$ 가 정확히 어떤 단어의 임베딩 위에 놓이도록 보장하므로 argmax 라운딩만으로 충분해야 한다. 하지만 경험적으로 모델은 단일 단어로 약속하는 $\mathbf{x}_0$ 를 만들어 내지 못한다.

이 현상의 한 설명은, 본 목적함수의 $\mathcal{L}_{\text{simple}}(\mathbf{x}_0)$ 항이 $\mathbf{x}_0$ 의 구조를 모델링하는 데 충분한 강조를 두지 않는다는 것이다. 정의를 떠올려 보면 $\mathcal{L}_{\text{simple}}(\mathbf{x}_0) = \sum_{t=1}^T \mathbb{E}_{\mathbf{x}_t} \|\mu_\theta(\mathbf{x}_t, t) - \hat{\mu}(\mathbf{x}_t, \mathbf{x}_0)\|^2$ 으로, 모델 $\mu_\theta(\mathbf{x}_t, t)$ 는 매 단계 $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 의 평균을 직접 예측한다. 이 목적함수에서 $\mathbf{x}_0$ 가 단일 단어 임베딩에 약속되어야 한다는 제약은 t 가 0 에 가까운 항에서만 나타난다. 이 매개변수화는 그 항을 강조하도록 세심한 튜닝 이 필요했다.

저자들의 접근은 $\mathcal{L}_{\text{simple}}$ 을 재매개변수화하여 Diffusion-LM 이 모든 항에서 $\mathbf{x}_0$ 를 명시적으로 모델링하게 한다. 구체적으로 $\mathbf{x}_0$ 로 매개변수화한 $\mathcal{L}_{\text{simple}}$ 의 유사물

$$\mathcal{L}_{\mathbf{x}_0\text{-simple}}^{\text{e2e}}(\mathbf{x}_0) = \sum_{t=1}^T \mathbb{E}_{\mathbf{x}_t} \|f_\theta(\mathbf{x}_t, t) - \mathbf{x}_0\|^2,$$

을 유도한다. 모델 $f_\theta(\mathbf{x}_t, t)$ 는 $\mathbf{x}_0$ 를 직접 예측한다. 이는 신경망에 매 항에서 $\mathbf{x}_0$ 를 예측하도록 강제하며, 이 목적함수로 학습한 모델은 $\mathbf{x}_0$ 가 정확히 어떤 단어 임베딩의 중심에 위치해야 함을 빠르게 학습한다.

재매개변수화는 학습에 도움을 주지만, 같은 직관을 디코딩에도 적용할 수 있다 — 저자들이 클램핑 트릭(clamping trick) 이라 부르는 기법. 표준 $\mathbf{x}_0$ 매개변수화 디코딩에서는 $\mathbf{x}_t$ 로부터 $\mathbf{x}_0$ 추정 $f_\theta(\mathbf{x}_t, t)$ 를 계산한 뒤 $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}} f_\theta(\mathbf{x}_t, t) + \sqrt{1 - \bar{\alpha}} \epsilon$ ($\bar{\alpha}_t = \prod_{s=0}^t (1 - \beta_s)$, $\epsilon \sim \mathcal{N}(0, I)$) 로 $\mathbf{x}_{t-1}$ 을 표집한다. 클램핑 트릭에서는 예측 벡터 $f_\theta(\mathbf{x}_t, t)$ 를 가장 가까운 단어 임베딩 시퀀스로 추가로 매핑한다. 표집 단계는 다음으로 바뀐다.

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}} \cdot \text{Clamp}(f_\theta(\mathbf{x}_t, t)) + \sqrt{1 - \bar{\alpha}} \epsilon.$$

클램핑 트릭은 중간 단계에서 예측 벡터가 단어로 약속하도록 강제하여, 벡터 예측을 더 정밀하게 만들고 라운딩 오류를 줄인다.

0.6 5. Diffusion-LM 의 디코딩과 제어 가능 생성

Diffusion-LM 을 정의했으니 이제 제어 가능 텍스트 생성 (§ 5.1) 과 디코딩 (§ 5.2) 을 다룬다.

0.6.1 5.1 제어 가능 텍스트 생성

Diffusion-LM 위에서 플러그앤플레이 제어를 가능케 하는 절차를 설명한다. 본 접근은 § 3.1 의 베이지 정형에서 영감을 받지만, 이산 텍스트에 직접 제어를 가하는 대신 Diffusion-LM 이 정의하는 연속 잠재 변수 시퀀스 $\mathbf{x}_{0:T}$

에 제어를 가하고, 라운딩으로 잠재를 텍스트로 변환한다.

$\mathbf{x}_{0:T}$ 제어는 사후 $p(\mathbf{x}_{0:T} | \mathbf{c}) = \prod_{t=1}^T p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{c})$ 로부터의 디코딩과 같다. 이 결합 추론을 매 단계의 제어 문제 시퀀스로 분해한다.

$$p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{c}) \propto p(\mathbf{x}_{t-1} | \mathbf{x}_t) \cdot p(\mathbf{c} | \mathbf{x}_{t-1}, \mathbf{x}_t).$$

선행 디퓨전 제어 연구의 조건부 독립 가정으로 $p(\mathbf{c} | \mathbf{x}_{t-1}, \mathbf{x}_t) = p(\mathbf{c} | \mathbf{x}_{t-1})$ 로 단순화한다. 결국 단계 t 에서 $\mathbf{x}_{t-1}$ 에 대해 그라디언트 업데이트를 돌린다.

$$\nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{c}) = \nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{x}_{t-1} | \mathbf{x}_t) + \nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{c} | \mathbf{x}_{t-1}).$$

두 항 모두 미분 가능 — 첫째 항은 Diffusion-LM, 둘째 항은 신경망 분류기로 매개변수화된다.

이미지 영역의 선행 연구들처럼, 분류기를 디퓨전 잠재 변수 위에서 학습하고 잠재 공간 $\mathbf{x}_{t-1}$ 에 대해 그라디언트 업데이트를 돌려 제어 만족 방향으로 조향한다. 이미지 디퓨전 연구는 보통 디퓨전 단계마다 $\nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{c} | \mathbf{x}_{t-1})$ 방향으로 한 번 그라디언트 스텝을 밟는다. 텍스트에서의 성능 향상과 디코딩 가속을 위해 본 연구는 두 가지 핵심 수정 — 유창성 정규화와 복수 그라디언트 스텝 — 을 도입한다.

유창한 텍스트를 생성하기 위해, 유창성 정규화가 적용된 제어 목적함수에 대해 그라디언트 업데이트를 돌린다.

$$\lambda \log p(\mathbf{x}_{t-1} | \mathbf{x}_t) + \log p(\mathbf{c} | \mathbf{x}_{t-1}),$$

λ 는 유창성(첫째 항)과 제어(둘째 항) 사이를 절충하는 하이퍼파라미터다. 기존 디퓨전 제어 가능 생성 방법은 $\lambda \log p(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 항을 포함하지 않지만, 이 항이 유창한 텍스트 생성에 결정적임이 관찰되었다. 제어 가능 생성 과정은 $p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{c})$ 의 최대화와 표집을 균형 잡는 확률적 디코딩으로 볼 수 있으며, 이는 nucleus 표집이나 저온도 표집과 비슷하다. 제어 품질을 높이기 위해 디퓨전 단계마다 여러 번 그라디언트 스텝을 밟는다 — 단계당 Adagrad 업데이트 3 회. 계산 비용을 보전하기 위해 디퓨전 단계를 2000→200으로 다운샘플링한다. 이는 표본 품질을 크게 해치지 않으면서 제어 알고리즘을 가속한다.

0.6.2 5.2 최소 베이지 위험 디코딩

기계 번역이나 문장 인필링처럼 단일 고품질 출력 시퀀스를 요구하는 조건부 텍스트 생성 과제가 많다. 이런 설정에서는 Diffusion-LM 으로부터 표본 집합 $\mathcal{S}$ 를 뽑은 뒤 손실 함수 $\mathcal{L}$ (예: 음의 BLEU) 아래 최소 기대 위험을 달성하는 표본을 고르는 최소 베이지 위험 (Minimum Bayes Risk, MBR) 디코딩을 적용한다.

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w} \in \mathcal{S}} \sum_{\mathbf{w}' \in \mathcal{S}} \frac{1}{|\mathcal{S}|} \mathcal{L}(\mathbf{w}, \mathbf{w}').$$

MBR 은 자주 고품질 출력을 돌려 준다 — 저품질 표본은 나머지와 비유사해서 손실 함수의 페널티를 받는다.

0.7 6. 실험 설정

학습 (§ 4) 과 디코딩 (§ 5) 의 개선 위에서, Diffusion-LM 을 두 언어모델링 과제로 학습한다. 그 뒤 5 개 분류기 가이드선 제어 과제에 제어 가능 생성 방법을 적용하고, 분류기 없는 제어 과제(인필링)에는 MBR 디코딩을 적용한다.

0.7.1 6.1 데이터셋과 하이퍼파라미터

Diffusion-LM 은 두 데이터셋으로 학습한다 — E2E (음식 종류·가격·고객 평점 등 8 개 필드로 라벨된 5만 건의 식당 리뷰) 와 ROCStories (98K 개의 5-문장 짧은 이야기, 일상적 사건 사이의 인과·시간 상식을 풍부히 담음). ROCStories 는 어휘 11K, 의미 콘텐츠가 다양해 모델링이 더 어렵다.

본 Diffusion-LM 은 트랜스포머 기반 80M 파라미터, 시퀀스 길이 $n = 64$, 디퓨전 단계 $T = 2000$, sqrt 노이즈 스케줄(부록 § A 참조). 임베딩 차원은 하이퍼파라미터 — E2E 는 $d = 16$, ROCStories 는 $d = 128$. 디코딩 시 E2E 는 200 단계로 다운샘플하고 ROCStories 는 2000 단계 유지. 200 단계로 디코딩해도 자기회귀 LM 보다 7 배 느리다. 제어 가능 생성에서 본 방법은 FUDGE 보다 1.5 배 느리지만 PPLM 보다 60 배 빠르다.

0.7.2 6.2 제어 과제

저자들은 6 개 제어 과제를 검토한다. 첫 4 과제는 분류기에 의존하고, 마지막 2 과제는 분류기 없이 가능 (길이는 Diffusion-LM 기반 방법에서만 분류기 없음). 각 과제마다 검증 분할에서 200 개 제어 표적 c 를 뽑고, 각 표적당 50 개 표본을 생성한다. 유창성 평가는 선행 연구를 따라 잘 파인튜닝된 GPT-2 (교사 LM)에 생성 텍스트를 넣어 lm -score (perplexity, 낮을수록 좋음)를 측정한다.

각 과제의 성공 지표는 다음과 같다.

- **Semantic Content** (의미 콘텐츠). 필드(예: rating)와 값(예: 5 star)이 주어지면 field=value 를 다루는 문장을 생성. value 의 정확 일치 성공률.
- **Parts-of-speech** (품사, POS). POS 태그 시퀀스(예: *Pronoun Verb Determiner Noun*)가 주어지면, 같은 길이의 단어 시퀀스 중 (오라클 POS 태거 기준) 태그가 표적과 일치하는 단어 수준 정확 일치율.
- **Syntax Tree** (구문 트리). 표적 구문 파스 트리가 주어지면, 즉시 사용 가능한 파서로 파싱한 결과의 F1 스코어.
- **Syntax Spans** (구문 스패). (스팬, 구문 카테고리) 쌍이 주어지면 그 스패의 파스가 표적과 일치하는 비율.
- **Length** (길이). 표적 길이 10, ..., 40 가 주어지면 ± 2 이내 길이의 시퀀스를 생성. Diffusion-LM 에서는 분류기 없이 다룬다.
- **Infilling** (인필링). aNLG 데이터셋의 좌 컨텍스트 O_1 , 우 컨텍스트 O_2 가 주어지면 두 컨텍스트를 논리적으로 잇는 문장 생성. Genie 리더보드의 자동·인간 평가를 보고.

0.7.3 6.3 분류기 가이드선 베이스라인

처음 5 과제에서 PPLM, FUDGE, 파인튜닝 오라클(FT)과 비교한다. PPLM·FUDGE 는 모두 자기회귀 LM 기반 플러그애플레이로, GPT-2 small 아키텍처로 처음부터 학습.

- **PPLM**: LM 활성화에 대한 그라디언트 상승으로 분류기·LM 확률을 동시에 키움. 단순 속성 제어에서 성공함. 위치 정보가 필요한 4 과제에는 PPLM 의 분류기에 위치 정보가 없어 적용하지 못해, semantic content 만

적용.

- FUDGE: 각 과제마다 미래 판별기(future discriminator)가 접두 시퀀스를 받아 완성 시퀀스의 제약 만족 여부를 예측. 디코딩 시 LM 예측을 판별기 점수로 재가중.
- FT (파인튜닝 오라클): 각 과제별로 (제어, 텍스트) 쌍으로 GPT-2 를 파인튜닝 — 플러그애플레이가 아닌 오라클 조건부 LM. 표집(temperature 1.0)과 빔 서치(빔 4) 출력을 각각 FT-sample, FT-search 로 보고.

0.7.4 6.4 인필링 베이스라인

인필링 과제에는 3 개 특화 베이스라인과 비교한다 — DELOREAN, COLD, AR-infilling (ROCStories 를 $(O_1, O_{middle}, O_2) \rightarrow (O_1, O_2, O_{middle})$ 으로 재정렬해 처음부터 학습한 자기회귀 LM).

0.8 7. 주요 결과

E2E 와 ROCStories 에서 Diffusion-LM 을 학습한다. NLL 면에서, Diffusion-LM 의 변분 상한은 동등 크기의 자기회귀 트랜스포머에 못 미친다 (E2E 2.28 vs 1.77, ROCStories 3.88 vs 3.05). 모델·데이터셋 크기를 키우면 격차가 부분적으로 줄어든다 (ROCStories 3.88 $\rightarrow$ 3.10). 우도가 더 나뉘어도, Diffusion-LM 기반 제어 가능 생성은 자기회귀 LM 기반보다 훨씬 좋은 출력을 만든다는 것을 § 7.1, § 7.2, § 7.3 에서 보일 것이다.

0.8.1 7.1 분류기 가이드스 제어 가능 텍스트 생성 결과

표 1 처럼 Diffusion-LM 은 모든 분류기 가이드스 과제에서 높은 성공률과 좋은 유창성을 달성하며, PPLM·FUDGE 베이스라인을 5 과제 모두에서 큰 폭 능가한다. 놀랍게도 구문 파스 트리·스팬 제어에서는 파인튜닝 오라클까지 능가하고, 나머지 3 과제에서는 비슷한 성능에 도달한다.

	Content (/lm)	POS	Syntax	Spans	Length
PPLM	9.9 / 5.32	–	–	–	–
FUDGE	69.9 / 2.83	27.0 / 7.96	17.9 / 3.39	54.2 / 4.03	46.9 / 3.11
Diffusion-LM	81.2 / 2.55	90.0 / 5.16	86.0 / 3.71	93.8 / 2.53	99.9 / 2.16
FT-sample	72.5 / 2.87	89.5 / 4.72	64.8 / 5.72	26.3 / 2.88	98.1 / 3.84
FT-search	89.9 / 1.78	93.0 / 3.31	76.4 / 3.24	54.4 / 2.19	100.0 / 1.83

(= ctrl 성공률 $\uparrow$, lm = lm-score $\downarrow$)

구문 파스 트리·스팬 제어가 파인튜닝에 까다로운 이유는 (1) 파스 트리에 조건화하려면 중첩 구조에 대한 추론이 필요하고, (2) 스팬 조건화에는 표적 위치에 올바른 구성소가 오도록 룩어헤드 계획(lookahead planning)이 필요하기 때문이다.

PPLM 은 의미 콘텐츠 제어에서 실패한다. 추측건대 PPLM 이 거친 속성 제어용으로 설계되어, 식당 리뷰가 Starbucks 를 언급해야 한다 같은 표적 과제에는 유용하지 않기 때문이다.

FUDGE 는 의미 콘텐츠 제어에서는 잘 작동하지만 나머지 4 과제에서 부진하다. 구조적 출력(POS, Syntax) 제어가 어려운 이유는 접두에서 한 번 실수하면 판별기가 모든 후속예에 낮은 확률을 부여하기 때문. 계획이 필요한

과제(Length, Spans)에서는 미래 판별기가 암묵적 룩어헤드를 학습해야 해 학습이 어렵다.

Diffusion-LM 의 *비자기회귀* 성격이 정밀한 미래 계획이 필요한 과제(Spans, Length)를 쉽게 풀게 한다. 전역 구조(POS, Syntax) 제어에서도 잘 작동하는 이유는, *거친-세밀 표현*이 분류기로 하여금 시퀀스 전역($t \approx T$)뿐 아니라 개별 토큰($t \approx 0$)에도 제어를 행사할 수 있게 하기 때문이라 본다.

정성 결과. § 부록 의 Syntax 제어 표본에서, 본 방법과 파인튜닝은 대체로 제약을 만족하는 유창한 문장을 만든 반면 FUDGE 는 첫 몇 단어 이후 제약을 벗어났다. 본 방법과 파인튜닝의 결정적 차이는, Diffusion-LM 은 실패한 스패를 보정하여 후속 스패를 표적에 맞출 수 있다는 점이다. 첫 예에서 생성된 스패 (“Family friendly Indian food”) 은 표적보다 한 단어 많아 실패하지만, Diffusion-LM 은 접속사를 떨어뜨려 보정해 후속에 오류가 전파되지 않는다. 둘째 예에서는 FT 모델이 (“The Mill”) 한 단어 적은 실패 스패를 만든 뒤, 보정에 실패해 후속 스패가 줄줄이 어긋난다.

0.8.2 7.2 제어 합성

플러그앤플레이의 고유한 능력 하나는 *모듈성*이다. 다중 독립 과제용 분류기가 주어진다면, 분류기 로그 확률의 합으로 그래디언트 가이드를 적용해 *교집합 제어*에서의 생성을 단순하게 만들 수 있다.

Semantic Content + Syntax, Semantic Content + POS 두 조합을 평가한다. 표 2 처럼 Diffusion-LM 은 양 구성요소 모두에서 높은 성공률을 달성하지만, FUDGE 는 더 전역적인 구문 제어를 포기한다 (단독 구문 제어에 실패하므로 예상 가능). 파인튜닝 모델은 POS·의미 콘텐츠를 *개별*로는 잘하지만, *product of experts(PoE)* 합성에서는 두 제약 모두에서 큰 성공률 하락을 보인다.

	Content+Syntax (sem / syn / lm)	Content+POS (sem / POS / lm)
FUDGE	61.7 / 15.4 / 3.52	64.5 / 24.1 / 3.52
Diffusion-LM	69.8 / 74.8 / 5.92	63.7 / 69.1 / 3.46
FT-PoE	61.7 / 29.2 / 2.77	29.4 / 10.5 / 2.97

0.8.3 7.3 인필링 결과

표 3 처럼 Diffusion-LM 은 연속 완화 기반 방법(COLD, DELOREAN)을 인필링에서 큰 폭 능가하며, 이 과제에 특화 학습된 모델 파인튜닝과 비교 가능한 성능에 도달한다. 자동 평가 점수가 약간 더 좋고, 인간 평가에서는 두 방법 사이에 통계적으로 유의한 차이가 없다. 이 결과는 Diffusion-LM 이 *생성 순서나 어휘 제약에 의존하는 다양한 제어 가능 생성 과제*를 특화 학습 없이도 풀 수 있음을 시사한다.

	BLEU-4 ↑	ROUGE-L ↑	CIDEr ↑	BERTScore ↑
Left-only	0.9	16.3	3.5	38.5
DELOREAN	1.6	19.1	7.9	41.7
COLD	1.8	19.5	10.7	42.7
Diffusion-LM	7.1	28.3	30.7	89.0
AR (인필링용 학습)	6.7	27.0	26.9	89.0

0.8.4 7.4 절제 연구

§ 4 의 설계 선택의 중요성을 두 절제로 확인한다. 표본 품질은 lm-score 로 측정.

- 학습 임베딩 vs 무작위 임베딩 (§ 4.1). ROCStories(어려운 LM 과제)에서 학습 임베딩이 무작위를 의미 있게 능가. E2E 에서는 같은 추세지만 격차가 작다.
- 목적함수 매개변수화 (§ 4.2). 모델이 x_0 를 직접 예측하도록 한다. 이미지 생성 표준의 ϵ 매개변수화와 비교하면, x_0 매개변수화는 차원 전반에서 일관되게 좋은 성능을 보이는 반면 ϵ 은 작은 차원에서만 작동하고 큰 차원에서 빠르게 붕괴한다.

0.9 8. 결론과 한계

저자들은 연속 디퓨전에 기반한 새 제어 가능 언어모델 Diffusion-LM 을 제안하여, 새로운 형태의 복잡 세밀 제어 과제를 가능케 했다. 6 개 세밀 제어 과제에서의 성공을 보였으며 — 본 방법은 기존 방법의 제어 성공률을 거의 두 배로 끌어올리고, 추가 학습이 필요한 파인튜닝 베이스라인과 경쟁적이다.

Diffusion-LM 이 가능케 하는 복잡 제어는 매력적이며, 이산 자기회귀 생성이라는 현재 패러다임으로부터의 실질적 이탈로 흥미롭다. 새 기술이 늘 그렇듯 단점도 있다 — (1) perplexity 가 더 높고, (2) 디코딩이 상당히 느리며, (3) 학습 수렴이 더 느리다. 후속 연구와 최적화로 이 문제들 다수가 해결될 수 있을 것이며, 이 접근이 대규모 제어 가능 생성의 매력적인 길이 될 것이라 본다.

0.10 부록 발췌 (요약)

원문 부록은 다음 내용을 다룬다.

- §A 노이즈 스케줄. 표준 스케줄(linear, cosine)은 텍스트에 잘 맞지 않는다. 텍스트의 이산성과 라운딩 단계 때문에 모델이 $t \approx 0$ 부근의 노이즈에 둔감해 — 단어 임베딩에 약간의 가우시안 노이즈를 더해도 최근점 이웃이 잘 안 바뀌어 잡음제거가 trivial해진다. 이를 해소하는 *sqr*t 스케줄 $\bar{\alpha}_t = 1 - \sqrt{t/T + s}$ ($s = 10^{-4}$, $T = 2000$) 을 제안. 처음 50 단계에서 노이즈를 빠르게 올리고 이후 완만히 진행해, 너무 어려운 고노이즈 문제에 단계를 낭비하지 않는다.
- §B 하이퍼파라미터. Diffusion-LM 디퓨전 단계 $T = 2000$, BERT-base 아키텍처, 시퀀스 길이 $n = 64$. $d \in \{16, 64, 128, 256\}$ 중 E2E $d = 16$, ROCStories $d = 128$. AdamW, 선형 감소 학습률 $1e-4$ 시작, 드롭 아웃 0.1, 배치 크기 64. E2E 200K 반복, ROCStories 800K 반복. 단일 A100 에서 200K 반복 ≈ 5 시간. $\mathcal{L}_{\text{vlb}}^{\text{e2e}}$ 안정화에는 그라디언트 클리핑 1.0 + 중요도 표집 ([nichol2021improved]) 필요. $\mathcal{L}_{\text{simple}}^{\text{e2e}}$ 에는 두 트릭 모두 불필요. 제어 가능 생성에서는 Adagrad, lr $\in \{0.05, 0.1, 0.15, 0.2\}$, 절충 $\lambda \in \{0.1, 0.01, 0.001, 0.0005\}$ 를 튜닝.
- §C 디코딩 속도. Diffusion-LM 은 $T = 2000$ 단계로 $O(2000)$ 모델 호출 (자기회귀 LM 은 $O(n)$). 짧은-중간 길이 시퀀스에서 디퓨전이 더 느리다. 50 시퀀스(길이 64) 디코딩에 ≈ 1 분. 2000 $\rightarrow$ 200 다운샘플은 E2E 에서 표본 품질 손실이 거의 없지만 ROCStories 에서는 손실이 있다. 50 표본 생성에 PPLM ≈ 80 분, FUDGE ≈ 50 초, Diffusion-LM ≈ 80 초.
- §D 분류기 정의. Semantic Content 분류기는 텍스트로 (필드, 값) 쌍을 예측하는 GPT-2 small 자기회귀 LM. POS 분류기는 BERT-base 기반 — 입력은 단어 임베딩의 결합, 출력은 단어별 POS 태그 소프트맥스. Syntax Tree 분류기는 트랜스포머 기반 구성소 파서로 각 스펠에 “비-구성소” 또는 라벨을 예측. Syntax Span 분류기는 같은 파서의 스펠-라벨 로그 확률을 사용.

- §E 엔드투엔드 목적함수 유도. $\mathcal{L}_{\text{vlb}} \rightarrow \mathcal{L}_{\text{simple}}$ 의 단순화를 따라 $\mathcal{L}_{\text{vlb}}^{\text{e2e}} \rightarrow \mathcal{L}_{\text{simple}}^{\text{e2e}}$ 을 유도. μ_θ 매개변수화 $\leftrightarrow x_0$ 매개변수화가 상수 스케일링만 다름을 보이고, 라운딩 오류 감소를 위해 모든 항에 x_0 매개변수화를 적용한 $\mathcal{L}_{x_0\text{-simple}}^{\text{e2e}}$ 을 도출.
- §F 로그 우도 모델·결과. § 4 의 학습 절차에서 몇 가지 변형 — 저차원 토큰 임베딩 시퀀스 대신 원-핫 토큰 벡터 시퀀스 직접 학습, [kingma2021variational] 처럼 연속 시간 디퓨전을 로그 우도 하한으로 학습하면서 노이즈 스케줄을 손실 분산 최소화로 동시에 학습, 출력에 소프트맥스를 두고 모든 제공오차 항을 교차 엔트로피로 대체, 입력에 $x := \text{softmax}(\alpha(t)x + \beta(t))$ 변환 (학습되는 MLP), 추론 시 라운딩 생략. 이런 우도 모델 개선은 본 실험에서 생성 품질 향상으로 이어지지는 않아, 본문은 원래 방법에 집중. NLL (nat/token) 결과는 다음과 같다.

	Small AR	Small Diffusion	Medium Diffusion
E2E	1.77	2.28	–
ROCStories	3.05	3.88	–
ROCStories (+GPT-J 합성 증강)	2.41	3.59	3.10

- §G-H 정성 표본·추가 절제. 비조건 표본(ROCStories+Aug, ROCStories, E2E), Syntax Span / POS / 길이 / 의미 콘텐츠 / Syntax Tree 제어의 정성 비교(FUDGE / Diffusion-LM / FT). 학습 임베딩 vs 무작위, sqrt vs cosine vs linear 스케줄, 트랜스포머 vs U-Net 아키텍처 추가 절제.
- §I 사회적 영향. 강한 제어성은 독성 완화·사실성 향상에 도움이 될 수 있으나, 표적화 허위 정보(narrative wedging 등)에 악용될 위험도 있다. 유창성을 해치지 않으면서 출력에 워터마크(watermark)를 삽입하는 생성 — 식별 가능성·유창성을 제약 조건으로 두는 제어 가능 생성 문제 — 을 향후 과제로 제안.

자세한 내용은 [arxiv 원문 PDF](#) 를 참고.